Computer Networks

GATE CSE 2027 · complete formula, rule, method & trap sheet

101 reference boxes · **9** chapters ·

Principles of Layering | Switching & Performance Metrics | Data Link Layer & MAC | Routing Algorithms | IPv4: Fragmentation, CIDR & NAT | TCP: Flow & Congestion Control | DNS & HTTP | Problem → Standard Method Playbook | Last-minute one-liners

1 Principles of Layering

Why layer at all? Core

NOTES

A network has to solve many different problems at once: turning bits into electrical signals, getting a frame across one wire, routing across the globe, making delivery reliable, and letting an app request a web page. Layering splits this into independent jobs — each layer offers a clean **service** to the layer above and uses services from the layer below, without needing to know how those services are implemented internally.

The TCP/IP stack (5 layers) Core

FLOW

Application → Transport → Network → Data Link → Physical

REFERENCE TABLE

Layer	Job	Data unit (PDU)	Example
Application	What the user's program actually wants to say	Message	HTTP request, DNS query
Transport	End-to-end delivery between processes (ports)	Segment (TCP) / Datagram (UDP)	TCP, UDP
Network	Routing across many networks, host-to-host	Packet (IP datagram)	IP, routers
Data Link	Reliable transfer across ONE link, framing, MAC addressing	Frame	Ethernet, Wi-Fi
Physical	Raw bits over a physical medium	Bits/symbols	Cables, radio signals

GATE FAVOURITE

GATE note

The classic 7-layer **OSI model** (with separate Session and Presentation layers) is still useful background, but the current syllabus focuses on the practical **TCP/IP 5-layer model** actually used on the real internet. Know that OSI's Session+Presentation+Application roughly collapse into TCP/IP's single Application layer.

Encapsulation & decapsulation Core

NOTES

As data goes DOWN the stack at the sender, each layer wraps the data from above inside its own header (and sometimes trailer) — this is **encapsulation**. At the receiver, data goes UP the stack and each layer strips off its own header — **decapsulation**.

KEY POINT

```
App data
  → [TCP header | App data]                    = Segment
    → [IP header | TCP header | App data]       = Packet
      → [Ethernet header | IP header | ... | Ethernet
trailer] = Frame
        → 1011010010110...                     = Bits on
the wire
```

Each layer only reads/writes its OWN header — it treats everything from the layer above as opaque data (this is exactly what "layering" buys you: a router only looks at the IP header, never at the TCP or application data inside).

WORKED EXAMPLE

Worked numerical

Q: An application sends 1000 bytes of data. TCP adds a 20-byte header, IP adds a 20-byte header, and Ethernet adds an 18-byte header+trailer. What is the total frame size placed on the wire, and what is the header overhead as a percentage?

A: Total = 1000 + 20 + 20 + 18 = **1058 bytes**. Overhead = 58 bytes → 58/1058 ≈ **5.5%** of the frame is pure header/trailer overhead.

Header sizes, MTU & MSS — the overhead arithmetic Core

REFERENCE TABLE

Layer	Header (typical)	Notes
TCP	20 B (up to 60 B with options)	UDP header is a fixed 8 B
IP (IPv4)	20 B (up to 60 B)	IHL counts 4-byte words; min 5
Ethernet	14 B header + 4 B CRC = 18 B	Payload 46–1500 B; **MTU = 1500 B**

KEY POINT

```
MSS = MTU − IP header − TCP header = 1500 − 20 − 20 = 1460
bytes of application data per segment
Efficiency = useful data / total bytes on the wire
Switches work up to layer 2 · routers up to layer 3 · hubs /
repeaters at layer 1
```

2 Switching & Performance Metrics

Three switching techniques Core

REFERENCE TABLE

Type	How it works	Pros	Cons
Circuit switching	A dedicated end-to-end path is reserved before data flows (e.g. old telephone network)	Guaranteed bandwidth, predictable delay once set up	Setup/teardown time; wastes capacity if the link sits idle
Packet switching	Data is split into packets, each routed independently ("store-and-forward"), no reservation	Efficient — links shared via statistical multiplexing	Variable, unpredictable delay (congestion, queuing)
Virtual circuit switching	A logical path is set up once (like circuit switching), but data still travels as separate packets along it	Combines fast, ordered forwarding with statistical sharing	Still needs setup signaling; used in ATM, MPLS, Frame Relay

Delay components — memorize these four Core

KEY POINT

```
Transmission delay = L / B      (L = packet size in bits, B = link bandwidth in bits/sec)
Propagation delay  = d / v      (d = distance, v = signal propagation speed in the medium)
Queuing delay      = time waiting in a router's buffer before transmission
Processing delay   = time a router/switch takes to examine the packet header
```

Total end-to-end delay for ONE hop = Transmission + Propagation + Queuing + Processing.

WORKED EXAMPLE

Worked numerical

Q: A 10 Mbps link carries a 5000-bit packet over a 2000 km cable where the signal travels at 2×10^8 m/s. Find the transmission delay, propagation delay, and total delay (ignore queuing/processing).
A:

```
Transmission delay = 5000 bits / (10×10⁶ bps) = 0.5 ms
Propagation delay  = (2000×1000 m) / (2×10⁸ m/s) = 10 ms
Total delay = 0.5 + 10 = 10.5 ms
```

Notice propagation dominates over long distances even at decent bandwidth — a classic GATE observation.

Store-and-forward delay across multiple hops (packet switching) Core

NOTES

In pure packet switching, a router must receive the **entire packet** before it can start forwarding it to the next hop (it can't forward partial packets).

FORMULA / RULE

Worked numerical

Q: A packet of L=6400 bits travels through 3 links (2 intermediate routers) to reach its destination, each link having bandwidth B=1 Mbps. Ignoring propagation delay, find the total time for the packet to reach the destination.
A: Each hop needs a full transmission time of L/B = 6400/(1×10⁶) = 6.4 ms before the NEXT hop can even begin (store-and-forward).

```
Total time (ignoring propagation) = number_of_links × (L/B) = 3 × 6.4 ms = 19.2 ms
```

General formula: for n links (n−1 intermediate routers), **Total ≈ n × (L/B) + Σ(propagation delays)**.

WORKED EXAMPLE

Worked numerical — circuit switching comparison

Q: The same data (6400 bits) is sent using circuit switching over the same 3-link path (B=1 Mbps per link), with a call setup time of 15 ms and negligible propagation delay. Compare total time to the packet-switched case above.
A: Once the circuit is set up, data flows straight through at the bottleneck rate without per-hop store-and-forward waiting:

```
Circuit switching total = setup time + transmission time = 15 + 6.4 = 21.4 ms
Packet switching total (from above) = 19.2 ms
```

Here packet switching is actually *faster* for a single message, because circuit switching pays a big upfront setup cost — this trade-off (setup overhead vs. per-hop store-and-forward overhead) is exactly what GATE likes to compare.

Bandwidth-Delay Product Core

KEY POINT

```
Bandwidth-Delay Product (BDP) = Bandwidth × RTT
```

This is the amount of data that can be "in flight" on the link at any instant — the size of the "pipe". It tells you the minimum window size (Part 3) needed to keep a link 100% utilized.

WORKED EXAMPLE

Worked numerical

Q: A link has bandwidth 100 Mbps and RTT 40 ms. Find the bandwidth-delay product, and the minimum window size (in bytes) needed to keep the link fully utilized.
A:

```
BDP = 100×10⁶ bps × 40×10⁻³ s = 4×10⁶ bits = 4,000,000 bits
    = 4,000,000 / 8 = 500,000 bytes = 500 KB
```

The sender needs a window of at least **500 KB** "in flight" at all times, or the link will sit idle waiting for acknowledgments — this is exactly why TCP's window size matters so much on high-bandwidth, high-latency ("long fat") links.

Multiplexing — sharing one link High-yield

REFERENCE TABLE

Technique	How the link is shared	Note
FDM (frequency-division)	Each channel gets its own frequency band	Guard bands waste bandwidth; analogue-style sharing
WDM	FDM for light: each channel uses its own wavelength on one fibre	Number of channels = fibre bandwidth / channel spacing
Synchronous TDM	Fixed time slots, assigned in rotation to each input whether or not it has data	An idle owner wastes its slot; used in circuit switching
Statistical TDM	Slots go only to inputs that have data (packet switching)	Better utilisation, but needs addressing and gives variable delay

WORKED EXAMPLE

Worked numerical

Q: A T1 line multiplexes 24 voice channels of 64 kbps. What is its line rate?
A: Each frame carries 24 × 8 = 192 data bits plus 1 framing bit = **193 bits**, sent 8000 times per second: 193 × 8000 = **1.544 Mbps**. The payload alone is 24 × 64 kbps = 1.536 Mbps; the extra 8 kbps is framing overhead.

3 Data Link Layer & MAC

Error detection Core

REFERENCE TABLE

Method	Idea	Catches
Parity bit	One extra bit makes the total number of 1's even (or odd)	Any single-bit error; misses even numbers of bit-flips
Checksum	Sum the data in fixed-size blocks, send the complement	Cheap, used in IP/UDP/TCP headers; weaker than CRC
CRC (Cyclic Redundancy Check)	Treat data as a binary polynomial, divide by a fixed generator polynomial (mod-2 division), send the remainder	Very strong — catches all burst errors up to the length of the remainder

WORKED EXAMPLE

Worked numerical — CRC

Q: Data = 1101011011, Generator = 10011 (a 4-bit CRC, so append 4 zeros). Find the CRC remainder and the transmitted frame.
A:

```
Message with 4 zeros appended: 11010110110000  (14 bits)
Divide by 10011 using mod-2 (XOR) long division — XOR with the generator
whenever the leading bit of the current window is 1, otherwise just shift:

 11010 (leading 1, XOR)         01001
  10011                   →
  -----
  01001

 → bring down next bit, window=10011 (leading 1, XOR)   → 00000
 → bring down next bit, window=00001 (leading 0, shift)  → 00001
 → bring down next bit, window=00010 (leading 0, shift)  → 00010
 → bring down next bit, window=00101 (leading 0, shift)  → 00101
 → bring down next bit, window=01011 (leading 0, shift)  → 01011
 → bring down next bit, window=10110 (leading 1, XOR)    → 00101
 → bring down next bit, window=01010 (leading 0, shift)  → 01010
 → bring down next bit, window=10100 (leading 1, XOR)    → 00111
 → bring down last bit, window=01110 (leading 0, shift)  → 01110

All 14 bits consumed. Remainder = last 4 bits of the final window = 1110
```

Transmitted frame = original data + remainder = `1101011011 1110` The receiver divides the received frame by the same generator; a **zero remainder** means no error was detected. (GATE typically gives you the full division to perform by hand — practice mod-2 long division carefully, it's just repeated XOR, no carries.)

Framing — marking where a frame starts and ends High-yield

REFERENCE TABLE

Method	How it works
Character (byte) count	A length field starts the frame — one corrupted count desynchronises everything after it
Byte stuffing	Frames delimited by FLAG (0x7E). If FLAG or ESC (0x7D) appears in the data, the sender inserts an ESC byte before it
Bit stuffing (HDLC)	Flag = `01111110`. After **five consecutive 1s** in the data, the sender inserts a **0**; the receiver deletes a 0 that follows five 1s

WORKED EXAMPLE

Worked numerical

Q: Bit-stuff the data `0111110111111011111`. Byte-stuff the data 7E 41 7D 42 (FLAG = 7E, ESC = 7D).
A:

```
Bit stuffing — insert 0 after every run of five 1s:
  0 11111 [0] 0 11111 [0] 1 0 11111 [0]   →
01111100111110101111110   (19 bits become 22 bits)
Byte stuffing — ESC before every FLAG/ESC byte in the data:
  7E 41 7D 42  →  7D 7E 41 7D 7D 42      (4 bytes become 6 bytes)
```

Stuffing overhead depends on the data: all-ones data gets one extra bit per five.

Medium Access Control (MAC) — who transmits when? Core

ALOHA

KEY POINT

Pure ALOHA
Transmit whenever you have data — no coordination at all. Throughput: `S = G·e^(-2G)`. Maximum at G=0.5: **S_max ≈ 18.4%**.

KEY POINT

Slotted ALOHA
Time is divided into slots; transmit only at slot boundaries. Throughput: `S = G·e^(-G)`. Maximum at G=1: **S_max ≈ 36.8%**.

WORKED EXAMPLE

Worked numerical

Q: In a slotted ALOHA system, the offered load G = 2. Find the throughput S.
A: S = G·e^(-G) = 2·e^(-2) = 2 × 0.1353 ≈ **0.271** (27.1% — notice this is LOWER than the peak at G=1, since too many stations are now colliding).

CSMA / CSMA-CD / CSMA-CA

REFERENCE TABLE

Protocol	Rule	Used in
CSMA (Carrier Sense)	Listen before transmitting ("listen before talk")	Basic wired LANs
CSMA/CD	Listen while transmitting; if a collision is detected, abort immediately and back off	Classic (wired) Ethernet
CSMA/CA	Can't reliably detect collisions (wireless), so try to AVOID them — random backoff before every transmission, optional RTS/CTS handshake	Wi-Fi (802.11)

COMMON TRAP

Why CSMA/CD needs a MINIMUM frame size
A transmitting station must still be sending when a collision's signal comes back, or it will never know the collision happened. This forces: `Transmission time ≥ 2 × Propagation time`.

WORKED EXAMPLE

Worked numerical — minimum Ethernet frame size

Q: A CSMA/CD LAN has bandwidth 10 Mbps, and the maximum distance between two stations is 2500 m, with signal speed 2×10^8 m/s. Find the minimum frame size needed.
A:

```
Propagation delay (one way) = 2500 / (2×10⁸) = 12.5 µs
Worst case: collision signal must travel there AND back → 2 × Tprop = 25 µs

Minimum transmission time = 2 × Tprop = 25 µs
Minimum frame size = Bandwidth × Minimum transmission time
                   = 10×10⁶ bps × 25×10⁻⁶ s = 250 bits ≈ 32 bytes
```

(Real Ethernet uses extra safety margin, giving the well-known 64-byte minimum frame size — but the derivation logic is exactly this "2 × Tprop" rule.)

Ethernet framing & addressing Core

KEY POINT

```
| Preamble | Dest MAC (6B) | Src MAC (6B) | Type (2B) | Data (46-1500B) | CRC (4B) |
```

Each device has a globally unique **48-bit MAC address** burned into its NIC. A **switch** forwards frames only out the port leading to the destination MAC (learned by watching source addresses), unlike a **hub** which blindly repeats to every port — this is why switches eliminate collisions entirely (each switch port is its own collision domain).

Bridges & learning switches High-yield

NOTES

A **transparent (learning) bridge** or switch builds a table **MAC address → port** automatically by looking at the **source** address of every arriving frame. For each frame it then decides based on the **destination**:

REFERENCE TABLE

Destination	Action
Known, on a **different** port	**Forward** only out that port
Known, on the **same** port the frame arrived on	**Filter** (discard) — the destination already saw it
Unknown unicast, or broadcast / multicast	**Flood** out every port except the arrival port

KEY POINT

Table entries age out (typically ~5 min) so moved hosts are relearned. Redundant links in a bridged LAN create loops and broadcast storms; the **Spanning Tree Protocol (STP)** blocks some ports so the active topology is a tree rooted at the bridge with the lowest ID.

Device	Collision domains	Broadcast domains
Hub	1 (all ports)	1
Bridge / switch with n ports	**n** (one per port)	1 (unless VLANs are used)
Router with n interfaces	n	**n**

Flow control at the link layer — sliding window protocols High-yield

NOTES

Let T_t = frame transmission time (L/B) and T_p = propagation delay, and define **$a = T_p / T_t$**. A sender that waits for each ACK idles for a whole round trip ($2T_p$), so keeping the link busy needs several frames in flight.

REFERENCE TABLE

Protocol	Sender / receiver window	Efficiency (utilisation)	Sequence numbers (k bits)	After a lost frame
Stop-and-wait	1 / 1	$1 / (1 + 2a)$	1 bit (0, 1)	Resend that frame after the timeout
Go-Back-N	N / 1	$N / (1 + 2a)$, capped at 1	$N \le 2^k - 1$	Resend the lost frame **and every later one**; receiver discards out-of-order frames
Selective Repeat	N / N	$N / (1 + 2a)$, capped at 1	$N \le 2^{k-1}$ (sequence space ≥ 2N)	Resend **only** the lost frame; receiver buffers out-of-order frames

KEY POINT

The window that keeps the pipe exactly full is **1 + 2a** frames. Efficiency can never exceed 1, so for $N \ge 1 + 2a$ the utilisation is 100%. If a question gives the round-trip time directly: efficiency = $N \cdot T_t / (T_t + RTT)$. ACK transmission time is ignored unless the question says otherwise.

WORKED EXAMPLE

Worked numerical

Q: A 1 Mbps link, 200 km long (signal speed 2×10^8 m/s), carries 1000-bit frames. Find the stop-and-wait efficiency and throughput, the window needed for full utilisation, and the sequence-number bits needed for Go-Back-N and for Selective Repeat at that window.
A: T_t = 1000 / 10^6 = 1 ms; T_p = 200 000 / (2×10^8) = 1 ms, so a = 1.

```
Stop-and-wait: η = 1/(1+2a) = 1/3 = 33.3%  →  throughput = 1 Mbps × 1/3 ≈ 333 kbps
Window for 100% = 1 + 2a = 3 frames
Go-Back-N : N + 1 = 4 sequence numbers  →  2 bits
Selective Repeat: 2N = 6 sequence numbers → 3 bits
```

WORKED EXAMPLE

Worked numerical

Q: T_t = 1 ms and T_p = 5 ms. What is the efficiency of Go-Back-N with N = 7?
A: a = 5, so 1 + 2a = 11. Efficiency = 7 / 11 ≈ **63.6%**. With N = 11 or more it would reach 100%.

GATE FAVOURITE

GATE trap

With **3-bit** sequence numbers Go-Back-N can use a window of **7**, but Selective Repeat only **4**. If the Selective Repeat window equals the whole sequence space, a retransmitted old frame cannot be told apart from a new one.

Error correction — Hamming code Extension

NOTES

For m data bits, r check bits are needed so that $2^r \ge m + r + 1$: m = 4 → r = 3 (the (7,4) code); m = 7 or 8 → r = 4. Check bits sit at the **power-of-two positions** 1, 2, 4, 8, … and each one gives even parity over the positions whose binary index contains that bit. After a single-bit error, the **syndrome** (the failing checks read as a binary number) equals the **position of the flipped bit**. In terms of **minimum Hamming distance d**: a code detects up to d − 1 errors and corrects up to ⌊(d − 1)/2⌋ (the (7,4) code has d = 3).

WORKED EXAMPLE

Worked numerical

Q: Encode data bits 1 0 1 1 with the (7,4) Hamming code (even parity), then locate the error if the received word is 0110111.
A: Positions 1…7 are p1 p2 d3 p4 d5 d6 d7, with d3 d5 d6 d7 = 1 0 1 1.

```
p1 covers 1,3,5,7: 1⊕0⊕1 = 0      p2 covers 2,3,6,7: 1⊕1⊕1 = 1      p4 covers 4,5,6,7: 0⊕1⊕1 = 0
Codeword = 0 1 1 0 0 1 1  (0110011)
Received 0110111: checks p1 → 1 (fails), p2 → 0 (ok), p4 → 1 (fails)
Syndrome = (p4 p2 p1) = 101 = 5  →  bit 5 flipped; correct it to get 0110011.
```

Channel capacity and line coding Extension

REFERENCE TABLE

Law	Formula	Use when
Nyquist	$C = 2\,B\,\log_2 L$ bps	Noiseless channel, L signal levels, bandwidth B Hz
Shannon	$C = B\,\log_2(1 + S/N)$ bps	Noisy channel; this is the **upper limit** whatever the signal levels

NOTES

Convert decibels first: $S/N = 10^{(SNR_{dB}/10)}$, so 30 dB means S/N = 1000. When a problem gives both limits, the achievable rate is the **smaller** of the two.

WORKED EXAMPLE

Worked numerical

Q: A 3 kHz telephone line has a 30 dB signal-to-noise ratio. What is its maximum data rate? What does Nyquist give for a noiseless 4 kHz channel with 8 levels?
A: S/N = 10^3 = 1000, so Shannon: C = 3000 × $\log_2(1001)$ ≈ 3000 × 9.97 ≈ **29.9 kbps**. Nyquist: C = 2 × 4000 × $\log_2$ 8 = 2 × 4000 × 3 = **24 kbps**.

KEY POINT

Line coding. Manchester encoding puts a transition in the middle of **every** bit and the **direction** of that transition carries the bit value (textbooks differ on which direction means 1, so follow the convention the question states), so the receiver can recover the clock, but the **signalling rate is twice the bit rate** — 10 Mbps Ethernet used a 20 Mbaud signal. NRZ uses one signal level per bit (baud rate = bit rate) but long runs of equal bits lose synchronisation.

4 Routing Algorithms

Modeling the network as a graph Core

NOTES

Routers = nodes, links = edges, link costs = edge weights (could be based on delay, hop count, or bandwidth). Both algorithms below try to find the **shortest-cost path** from every router to every other router.

Distance Vector Routing (Bellman-Ford based) Core

NOTES

Each router only knows the cost to its **direct neighbors**. It periodically shares its entire distance table with its neighbors, and each router updates its own table using the **Bellman-Ford equation**:

KEY POINT

```
D_x(y) = min over all neighbors v of { c(x,v) + D_v(y) }
```

"My best cost to Y = my link cost to neighbor V, plus V's own best-known cost to Y — minimized over every neighbor V."

WORKED EXAMPLE

Worked numerical

Q: 3 routers A, B, C. Direct link costs: A-B=2, B-C=1, A-C=7. Using distance vector routing, what does A's table converge to for reaching C?
A:

```
Initial: A's table — A-B=2, A-C=7 (direct link only)
B shares its table with A: B says "my cost to C = 1"
A recomputes: D_A(C) = min( c(A,C)=7 ,  c(A,B)+D_B(C) =
2+1=3 ) = min(7,3) = 3

A updates its table: cost to C via B = 3 (better than the
direct 7-cost link)
Next hop for C, from A's table: via B
```

This is exactly how distance vector routing "discovers" that going through B is cheaper than A's own direct link to C — information propagates hop by hop until everyone converges.

COMMON TRAP

Count-to-infinity problem

If a link fails, bad news (increased cost) travels much slower than good news, because routers can keep telling each other stale, now-wrong "shortcuts" back and forth, incrementing the cost by tiny amounts each round instead of realizing the path is broken. Fixes: **split horizon** (don't advertise a route back to the neighbor you learned it from) and **poison reverse**.

Link State Routing (Dijkstra based) Core

NOTES

Every router **floods** its direct link costs to the ENTIRE network (not just neighbors), so every router ends up with a full map of the topology. Each router then independently runs **Dijkstra's algorithm** on this map to compute shortest paths to everyone.

WORKED EXAMPLE

Worked numerical — Dijkstra's trace

Q: Graph: A-B=4, A-C=1, C-B=2, C-D=5, B-D=1. Find shortest paths from A to every other node.
A:

```
Initialize: dist[A]=0, dist[B]=∞, dist[C]=∞, dist[D]=∞.
Visited={}

Round 1: pick unvisited min-dist node = A (dist 0). Visit A.
  Update neighbors: dist[B]=min(∞,0+4)=4,
dist[C]=min(∞,0+1)=1

Round 2: pick unvisited min-dist = C (dist 1). Visit C.
  Update neighbors: dist[B]=min(4, 1+2=3)=3 (better via C!)
                    dist[D]=min(∞, 1+5=6)=6

Round 3: pick unvisited min-dist = B (dist 3). Visit B.
  Update neighbors: dist[D]=min(6, 3+1=4)=4 (better via B!)

Round 4: pick unvisited min-dist = D (dist 4). Visit D.
Done.

Final shortest distances from A:  B=3 (path A-C-B),  C=1
(direct),  D=4 (path A-C-B-D)
```

Distance Vector vs Link State

REFERENCE TABLE

	Distance Vector	Link State
Info shared	Entire distance table, only to neighbors	Only own link costs, flooded to everyone
Each router knows	Only costs, not the full topology	The full network topology
Algorithm used locally	Bellman-Ford (distributed)	Dijkstra (run locally by each router)
Convergence issue	Count-to-infinity	Faster, more robust convergence, but more messaging/flooding overhead

Real routing protocols — RIP, OSPF, BGP Extension

REFERENCE TABLE

	RIP	OSPF	BGP
Scope	Interior (within an AS)	Interior (within an AS)	**Exterior** (between ASes)
Algorithm	Distance vector (Bellman-Ford)	Link state (Dijkstra)	**Path vector**
Metric	Hop count; **16 = infinity** (max usable 15)	Link cost (e.g. based on bandwidth)	Policy-based; AS-path length is one factor
Transport	UDP, port 520	Directly over IP (protocol 89)	**TCP**, port 179
Updates	Whole table to neighbours every ~30 s	LSAs flooded on change; hello packets	Incremental updates over a persistent session

KEY POINT

• RIP limits the network diameter to 15 hops precisely so that count-to-infinity terminates quickly (and uses split horizon / poison reverse).
• OSPF splits a big AS into **areas** joined by the backbone area 0, reducing flooding; it converges faster than RIP.
• BGP carries the full **AS path**, so a router rejects any route whose path already contains its own AS — this prevents loops without count-to-infinity. Routing policy (business relationships) matters more than shortest path.

5 IPv4: Fragmentation, CIDR & NAT

Fragmentation Core

NOTES

Every link has a **Maximum Transmission Unit (MTU)** — the largest frame it can carry. If a datagram is bigger than the next link's MTU, a router splits it into **fragments**. Each fragment gets its own IP header, with three key fields: **Identification** (same for all fragments of one datagram), **Fragment Offset** (position of this fragment's data, in units of **8 bytes**), and the **MF (More Fragments) flag** (1 for all fragments except the last).

WORKED EXAMPLE

Worked numerical

Q: A 4000-byte IP datagram (20-byte header + 3980-byte data) must cross a link with MTU = 1500 bytes. How many fragments are needed, and what is each fragment's offset?

A:

```
Data per fragment (excluding the 20-byte header each
fragment needs) must be
a multiple of 8 bytes, and at most 1500-20=1480 bytes → use
1480 bytes/fragment.

Total data to carry = 3980 bytes
Fragment 1: data bytes 0-1479    (1480 bytes) → offset = 0/8
= 0,      MF=1
Fragment 2: data bytes 1480-2959 (1480 bytes) → offset =
1480/8 = 185, MF=1
Fragment 3: data bytes 2960-3979 (1020 bytes) → offset =
2960/8 = 370, MF=0 (last)

Total fragments = 3
```

Each fragment is now (20-byte header + its data) ≤ 1500 bytes, satisfying the MTU.

CIDR (Classless Inter-Domain Routing) Core

NOTES

Instead of rigid class A/B/C blocks, CIDR writes an address as `a.b.c.d/n`, where **n** is the number of leading bits that form the network portion (the "subnet mask" length).

KEY POINT

```
Number of host addresses in a /n block = 2^(32-n)
Usable hosts (excluding network & broadcast addresses) =
2^(32-n) - 2
```

WORKED EXAMPLE

Worked numerical

Q: Given the block 192.168.10.0/26, find the network address, broadcast address, and number of usable hosts.

A:

```
/26 means 26 network bits, 32-26 = 6 host bits → 2^6 = 64
total addresses
Network address = 192.168.10.0     (all host bits = 0)
Broadcast address = 192.168.10.63  (all host bits = 1, since
64-1=63)
Usable hosts = 64 - 2 = 62
```

WORKED EXAMPLE

Worked numerical — subnetting

Q: You are given 192.168.1.0/24 and must split it into 4 equal-sized subnets. Find each subnet's CIDR block.

A:

```
4 subnets needs 2 extra bits (2² = 4) → new prefix = /24+2 =
/26 (64 addresses each)
Subnet 1: 192.168.1.0/26     (.0  to  .63)
Subnet 2: 192.168.1.64/26    (.64 to  .127)
Subnet 3: 192.168.1.128/26   (.128 to .191)
Subnet 4: 192.168.1.192/26   (.192 to .255)
```

NAT (Network Address Translation) Core

NOTES

Private networks use reserved address ranges (like `192.168.0.0/16`, `10.0.0.0/8`) that are never routed on the public internet. A NAT router at the network's edge rewrites the **(private IP, private port)** of outgoing packets to a single **(public IP, chosen port)**, keeping a translation table so replies can be routed back correctly.

KEY POINT

```
NAT Translation Table
Private side              ⇄   Public side
192.168.1.5 : 5000        ⇄   203.0.113.9 : 40001
192.168.1.6 : 5000        ⇄   203.0.113.9 : 40002
```

Two internal hosts using the SAME private port (5000) are distinguished on the public side purely by the different port NAT assigns them (40001 vs 40002) — this is exactly why NAT lets one public IP serve an entire private network.

IP support protocols — ARP, DHCP, ICMP Extension

ARP (Address Resolution Protocol)

NOTES

Maps a next-hop **IP address** → **MAC address** on the same LAN. The sender **broadcasts** an ARP request (destination MAC FF:FF:FF:FF:FF:FF: "who has IP x?"); the owner replies by **unicast**. Answers are kept in an **ARP cache**.

COMMON TRAP

GATE classic

Going from host H1 to a host on **another** network, H1 ARPs for the **default router's** interface, not for the final destination. As a packet crosses each router the **MAC addresses change at every hop**, but the **source/destination IP addresses stay the same** (unless NAT rewrites them).

DHCP (Dynamic Host Configuration Protocol)

KEY POINT

Gives a booting host its IP address, subnet mask, default gateway and DNS server, as a time-limited **lease**. Runs over **UDP** (server port 67, client port 68). The four steps — **DORA**:

```
1. DISCOVER  client → broadcast (src 0.0.0.0, dst
255.255.255.255)
2. OFFER     server → proposes an address
3. REQUEST   client → broadcast, accepting one offer
4. ACK       server → confirms the lease
```

A **relay agent** forwards broadcasts to a DHCP server on another network.

ICMP (Internet Control Message Protocol)

KEY POINT

Carried inside IP datagrams (**protocol number 1**); reports errors and supports diagnostics: **Echo request/reply** (ping), **Destination unreachable**, **Time exceeded** (TTL reached 0), **Redirect**. **traceroute** sends probes with TTL = 1, 2, 3 … ; each router that drops one returns "Time exceeded", revealing the path hop by hop. ICMP never generates errors about ICMP error messages.

Classful addressing and special addresses Extension

REFERENCE TABLE

Class	First octet	Default mask	Usable hosts per network
A	0 – 127 (1 – 126 usable; 127 is loopback)	/8	$2^{24} - 2 = 16{,}777{,}214$
B	128 – 191	/16	$2^{16} - 2 = 65{,}534$
C	192 – 223	/24	$2^{8} - 2 = 254$
D	224 – 239	— (multicast)	—
E	240 – 255	— (reserved)	—

KEY POINT

Special addresses. `127.0.0.0/8` loopback · `0.0.0.0` "this host" (e.g. a DHCP client before it has an address) · `255.255.255.255` **limited broadcast** (never forwarded by routers) · host bits all 1 = **directed broadcast** for that network · host bits all 0 = the network address · `224.0.0.0/4` multicast · private ranges `10.0.0.0/8`, `172.16.0.0/12`, `192.168.0.0/16` · `169.254.0.0/16` link-local (self-assigned when DHCP fails).

Forwarding: longest-prefix match, and VLSM Core

NOTES

A router compares the destination with **every** table entry and, among the entries that match, forwards by the one with the **longest prefix** (the most specific route). Subnetting with **variable-length masks (VLSM)** gives each subnet only as many addresses as it needs: allocate the **largest** subnet first and keep every block aligned to its own size.

WORKED EXAMPLE

Worked numerical

Q: A router has entries 192.168.0.0/16, 192.168.5.0/24 and 192.168.5.128/25. Which entry is used for 192.168.5.130, 192.168.5.20 and 192.168.9.9?
A: 192.168.5.130 matches all three (it lies in .128–.255) → the **/25** wins. 192.168.5.20 is outside the /25 but inside the /24 → the **/24**. 192.168.9.9 matches only the /16 → the **/16**.

WORKED EXAMPLE

Worked numerical — VLSM

Q: Split 192.168.1.0/24 for three LANs needing 100, 50 and 20 hosts.
A: Add 2 for the network and broadcast addresses and round up to a power of two:

```
100 hosts → 102 → 128 addresses → /25 : 192.168.1.0/25
(.0   – .127, 126 usable)
 50 hosts →  52 →  64 addresses → /26 : 192.168.1.128/26
(.128 – .191,  62 usable)
 20 hosts →  22 →  32 addresses → /27 : 192.168.1.192/27
(.192 – .223,  30 usable)
```

Addresses .224 – .255 stay free for later.

IPv4 header — fields worth knowing Core

REFERENCE TABLE

Field	Size / meaning
Version, IHL	4 bits each. IHL counts 4-byte words, so the header is 20 – 60 bytes
Total length	16 bits → a datagram is at most **65,535 bytes**
Identification, flags (DF, MF), fragment offset	Reassembly; the offset is 13 bits in units of 8 bytes
TTL	Decremented by each router; at 0 the packet is dropped and ICMP "time exceeded" is sent
Protocol	Names the payload: **1** = ICMP, **6** = TCP, **17** = UDP
Header checksum	Covers the **header only**, and is recomputed at every router because TTL changes

IPv6 in brief Extension

KEY POINT

128-bit addresses written as 8 groups of 4 hex digits; leading zeros may be dropped and **one** run of all-zero groups may be replaced by `::` (2001:0db8:0000:0000:0000:ff00:0042:8329 → `2001:db8::ff00:42:8329`). The header is a fixed **40 bytes** with **no header checksum** and **no fragmentation by routers** (only the source fragments, after path-MTU discovery). There is **no broadcast**: multicast and anycast replace it, and Neighbour Discovery (inside ICMPv6) replaces ARP. Migration uses **dual stack** or **tunnelling** (IPv6 packets carried inside IPv4).

6 TCP: Flow & Congestion Control

Flow control — sliding window Core

NOTES

The receiver advertises a **receive window (rwnd)**: how many unacknowledged bytes it can currently buffer. The sender must never have more than `min(cwnd, rwnd)` bytes "in flight" (sent but not yet acknowledged).

WORKED EXAMPLE

Worked numerical

Q: Sender's window size is 4000 bytes, MSS (max segment size) is 1000 bytes, RTT is 100 ms. What is the maximum achievable throughput?
A: The sender can have 4000 bytes in flight per RTT (send, wait for ack, then send more): `Throughput = Window size / RTT = 4000 bytes / 0.1 s = 40,000 bytes/s = 320 kbps`.

Congestion control — slow start & AIMD Core

KEY POINT

Slow Start

`cwnd` starts at 1 MSS and **doubles every RTT** (exponential growth) until it hits `ssthresh`, or a loss occurs.

KEY POINT

Congestion Avoidance (AIMD)

Once past ssthresh, `cwnd` grows by just **+1 MSS per RTT** (linear, additive increase). On a loss, `ssthresh = cwnd/2` and `cwnd` drops (multiplicative decrease).

WORKED EXAMPLE

Worked numerical — cwnd trace

Q: ssthresh = 16 MSS. Starting from cwnd=1 MSS, trace cwnd for the first several RTTs assuming no losses.

A:

```
Slow start (doubles each RTT, until reaching ssthresh=16):
RTT1: 1   RTT2: 2   RTT3: 4   RTT4: 8   RTT5: 16  (=
ssthresh, switch phase here)

Congestion avoidance (adds +1 MSS each RTT from here on):
RTT6: 17   RTT7: 18   RTT8: 19 ...
```

If a timeout (loss) occurred at, say, cwnd=19: new ssthresh = 19/2 ≈ 9, and cwnd resets to 1, restarting slow start up to the new ssthresh=9 before switching back to linear growth.

Socket API — the basic flow Core

KEY POINT

Server

`socket()` → `bind()` → `listen()` → `accept()` (blocks until a client connects) → `recv()/send()` → `close()`

KEY POINT

Client

`socket()` → `connect()` (initiates the 3-way handshake) → `send()/recv()` → `close()`

GATE FAVOURITE

GATE note

The 3-way handshake itself (SYN → SYN-ACK → ACK) is what `connect()` triggers on the client and what `accept()` completes on the server — remember it costs **1.5 RTT** before any actual data can be sent.

TCP segment header & UDP High-yield

KEY POINT

```
| Source port 16 | Dest port 16 |
| Sequence number 32 |
| Acknowledgment number 32 |
| HLEN 4 | Reserved | URG ACK PSH RST SYN FIN | Window 16 |
| Checksum 16 | Urgent pointer 16 | Options (0–40 B) |
```

Header = 20 B minimum, 60 B maximum (HLEN counts 4-byte words, range 5–15). **UDP** is connectionless with an 8-byte header: source port, destination port, length, checksum — no sequencing, retransmission or flow control.

Sequence & acknowledgment numbers, connection setup and teardown High-yield

KEY POINT

Sequence number = byte-stream number of the **first** data byte in the segment. **ACK number** = the **next byte expected** (cumulative). **SYN and FIN each consume one sequence number**; a pure ACK consumes none.

```
Setup (3-way):   Client → SYN, seq = x
                 Server → SYN+ACK, seq = y, ack = x + 1
                 Client → ACK, seq = x + 1, ack = y + 1
(data may ride with this segment)
Teardown (4-way): FIN → ACK → FIN → ACK  (each direction is
closed separately — "half-close")
The side that closes first enters TIME_WAIT for 2 × MSL
before releasing the connection.
```

COMMON TRAP

Remember

Dropped, reordered or lost data makes the receiver repeat the **same ACK number** (**duplicate ACK**). **Three duplicate ACKs** trigger **fast retransmit** of the missing segment without waiting for the timer. Tahoe then restarts at cwnd = 1; Reno (fast recovery) sets ssthresh = cwnd/2 and cwnd = ssthresh (+3). A **timeout** always drops cwnd to 1 MSS.

RTT estimation and the retransmission timeout High-yield

KEY POINT

```
SRTT   = (1 − α)·SRTT   + α·SampleRTT          α = 1/8
RTTVAR = (1 − β)·RTTVAR + β·|SRTT − SampleRTT|   β = 1/4
(computed with the OLD SRTT, as in RFC 6298)
RTO    = SRTT + 4·RTTVAR
```

Karn's algorithm: never take an RTT sample from a **retransmitted** segment (the ACK is ambiguous), and **double the RTO** on every timeout (exponential back-off).

Tahoe vs Reno — reacting to loss Core

REFERENCE TABLE

Event	TCP Tahoe	TCP Reno
Timeout	ssthresh = cwnd/2, cwnd = 1, slow start	Same
3 duplicate ACKs	Fast retransmit, then ssthresh = cwnd/2 and **cwnd = 1**	Fast retransmit, ssthresh = cwnd/2, **fast recovery**: cwnd = ssthresh + 3, +1 per further duplicate ACK, and when the new ACK arrives cwnd = ssthresh (back to congestion avoidance)

WORKED EXAMPLE

Worked numerical

Q: cwnd = 20 MSS when three duplicate ACKs arrive. Give ssthresh and cwnd for Tahoe and Reno. What if it had been a timeout?

A: ssthresh = 20/2 = **10** in both. Tahoe restarts at **cwnd = 1**. Reno sets **cwnd = 10 + 3 = 13**, inflates it during recovery, then drops to **10** when the new ACK arrives. On a **timeout** both set ssthresh = 10 and cwnd = 1.

Small segments: Nagle, silly window syndrome, delayed ACK Extension

KEY POINT

The problem: sending 1 byte of data costs 20 + 20 header bytes, so only 1/41 ≈ **2.4%** of the packet is useful.

Nagle's algorithm (sender): send a small segment immediately only if there is **no unacknowledged data** in flight; otherwise buffer small writes until an ACK arrives or a full MSS has accumulated.

Silly window syndrome (receiver): a slow application reads a few bytes at a time and the receiver keeps advertising tiny windows. **Clark's fix:** do not advertise a new window until it can hold a full MSS (or half the buffer).

Delayed ACK: the receiver waits (at most about **500 ms**) hoping to piggyback the ACK on data, and ACKs at least **every second full-sized segment**.

Window limits and window scaling Core

NOTES

The TCP window field is 16 bits, so the largest window is **65,535 bytes**. At RTT = 100 ms this caps throughput at 65 535 × 8 / 0.1 ≈ **5.24 Mbps** however fast the link is. The **window-scale option** (negotiated in the SYN, shift up to 14) multiplies the window by 2^{shift}, giving up to 65 535 × 2^{14} ≈ **1 GB**. This is the bandwidth-delay product again: the window must be at least BDP = bandwidth × RTT to keep the pipe full.

Traffic shaping — leaky bucket and token bucket Extension

REFERENCE TABLE

	Leaky bucket	Token bucket
Output	Constant rate ρ, however bursty the input	Bursts allowed, up to the tokens saved
Mechanism	A queue (buffer) drained at a fixed rate; overflow is dropped	Tokens arrive at rate r into a bucket of size b; sending a bit uses a token
Idle periods	Capacity is lost	Credit accumulates, up to b

WORKED EXAMPLE

Worked numerical — token bucket

Q: Token rate r = 2 Mbps, bucket size b = 1 Mb (full), line rate M = 10 Mbps. How long can the sender burst at full speed, and how much does it send?
A: During a burst of length S the bucket gives b tokens and r·S more arrive: M·S = b + r·S, so `S = b / (M − r)` = 1 / (10 − 2) = **0.125 s**. Data sent = 10 Mbps × 0.125 s = **1.25 Mb**, which equals 1 + 2 × 0.125 tokens ✓.

WORKED EXAMPLE

Worked numerical — leaky bucket

Q: Output rate 2 Mb/s, buffer 3 Mb, input a steady 6 Mb/s for 1 s. How much is dropped?
A: The queue grows at 6 − 2 = 4 Mb/s and fills after 3/4 = 0.75 s. For the last 0.25 s the excess (6 − 2 = 4 Mb/s) cannot be stored: 4 × 0.25 = **1 Mb dropped**.

Port numbers and common services High-yield

REFERENCE TABLE

Range	Name
0 – 1023	Well-known (needs privilege to bind)
1024 – 49151	Registered
49152 – 65535	Dynamic / ephemeral (client side)

NOTES

Common ports: FTP 20/21 · SSH 22 · Telnet 23 · SMTP 25 · DNS 53 (UDP for queries, TCP for zone transfers) · DHCP 67/68 (UDP) · HTTP 80 · POP3 110 · IMAP 143 · SNMP 161 (UDP) · HTTPS 443. Both TCP and UDP checksums also cover a **pseudo-header** (source and destination IP, protocol, length) in addition to the segment itself.

7 DNS & HTTP

DNS — Domain Name System Core

NOTES

DNS is a distributed, hierarchical database mapping domain names to IP addresses, organized in levels: **Root servers** → **TLD servers (.com, .org)** → **Authoritative servers** (for the specific domain).

FLOW

Client asks local resolver → Root server → TLD server (.com) → Authoritative server → IP address returned

REFERENCE TABLE

Query type	Behavior
Iterative	Each server gives the resolver the address of the NEXT server to ask — resolver does all the legwork
Recursive	The resolver asks one server, which itself goes and asks the rest, returning only the final answer

WORKED EXAMPLE

Worked numerical

Q: A DNS lookup requires visiting the local resolver, then a root server, then a TLD server, then the authoritative server (fully iterative, no caching). If each round-trip takes 20 ms, find the total resolution time.
A: 3 iterative round trips (resolver↔root, resolver↔TLD, resolver↔authoritative) = 3 × 20 = **60 ms**, plus the client's own request to the resolver and its reply (another 20 ms round trip) = **80 ms total**. (Caching at the resolver — thanks to each record's **TTL** — is exactly what avoids repeating this full chain on every single lookup.)

HTTP Core

REFERENCE TABLE

Method	Purpose
GET	Retrieve a resource
POST	Submit data to be processed (e.g. a form)
PUT	Replace a resource entirely
DELETE	Remove a resource

NOTES

Status codes: **2xx** success (200 OK), **3xx** redirection (301 Moved), **4xx** client error (404 Not Found), **5xx** server error (500 Internal Error).

KEY POINT

Non-persistent HTTP
A brand-new TCP connection (with its own 3-way handshake) for EVERY object on a page.

KEY POINT

Persistent HTTP
One TCP connection stays open and is reused for multiple objects/requests — avoids repeated handshake overhead.

WORKED EXAMPLE

Worked numerical

Q: A webpage has 1 HTML file plus 5 embedded images, all on the same server. RTT = 50 ms (assume negligible transmission time). Compare total time using non-persistent HTTP (no parallel connections) vs. persistent HTTP without pipelining (one connection, one request at a time).
A:

```
Non-persistent (serial, new connection each time):
  Each object = 1 RTT for TCP handshake + 1 RTT for HTTP
request/response = 2 RTT
  6 objects (1 HTML + 5 images) × 2 RTT = 12 RTT = 12 × 50 =
600 ms

Persistent, no pipelining (1 handshake total, then one
request/response at a time):
  1 RTT for the single handshake + 6 RTT for the 6
request/response exchanges
  = 7 RTT = 7 × 50 = 350 ms
```

With **pipelining** (all requests sent without waiting for each response) the six objects need only 1 RTT (handshake) + 1 RTT (HTML) + 1 RTT (all 5 images together) = 3 RTT = 150 ms. Persistent HTTP saves a huge amount of time by paying the handshake cost only **once** — this exact "count the RTTs" calculation is a favorite GATE question type.

Email and file transfer — SMTP, POP3/IMAP, FTP Extension

REFERENCE TABLE

Protocol	Port	Role
SMTP	TCP 25	**Push**: user agent → sender's mail server, and server → recipient's mail server (commands HELO, MAIL FROM, RCPT TO, DATA)
POP3	TCP 110	**Pull**: recipient downloads mail (usually deleting it from the server)
IMAP	TCP 143	**Pull**: mail stays on the server, folders managed remotely
FTP	TCP 21 (control) + 20 (data)	Separate **control** and **data** connections ("out-of-band" control); stateful

KEY POINT

Mail path: sender's UA —SMTP→ sender's server —SMTP→ recipient's server —POP3/IMAP→ recipient's UA. SMTP is **never** used to retrieve mail. HTTP is stateless and uses port 80 (443 for HTTPS); DNS uses port 53 (UDP for ordinary queries, TCP for zone transfers).

DNS resource records Core

REFERENCE TABLE

Type	Meaning
A / AAAA	Name → IPv4 / IPv6 address
NS	Name of the authoritative name server for a domain
CNAME	Alias → canonical name
MX	Mail server for a domain (with a preference value)
PTR	Reverse lookup: IP address → name

HTTP time with pipelining Core

KEY POINT

Base page + n embedded objects on the same server (RTT = R, transmission time negligible):

```
Non-persistent, serial :  (n + 1) × 2R          (handshake + request for each object)
Persistent, no pipeline:  R + (n + 1) × R        (one handshake, then one RTT per object)
Persistent, pipelined  :  R + R + R = 3R         (handshake, base page, then ALL objects together)
```

Cookies, conditional GET and web caching Core

KEY POINT

HTTP is stateless — the server remembers nothing between requests. **Cookies** add state: the server sends `Set-Cookie: id=…` in a response, and the browser returns `Cookie: id=…` on every later request to that site, which is how logins and shopping carts persist.

Conditional GET: a cache that already holds an object asks `If-Modified-Since: <date>`. If it has not changed, the server answers **304 Not Modified** with **no body**, saving a full transfer. A **web proxy cache** serves repeat requests locally, cutting both response time and the load on the access link.

WORKED EXAMPLE

Worked numerical

Q: A cache answers 40% of requests in 10 ms; the rest go to the origin server and take 200 ms. What is the average response time?
A: 0.4 × 10 + 0.6 × 200 = 4 + 120 = **124 ms** (versus 200 ms with no cache).

GATE FAVOURITE

GATE note

Status codes: **200** OK, **301** moved permanently, **304** not modified, **404** not found, **500** server error. HTTP/1.0 is non-persistent by default; HTTP/1.1 is **persistent** by default and supports pipelining.

8 Problem → Standard Method Playbook

METHOD

Framing / bit stuffing

```
Bit stuffing: after five consecutive 1s in the data, insert a 0.
At the receiver, remove the stuffed 0 after five 1s.
Byte stuffing: reserve an ESC byte and escape occurrences of the delimiter/ESC inside payload.
```

Trap: bit stuffing is bit-oriented; byte stuffing is byte/character-oriented.

METHOD

IP fragmentation / CIDR

```
Prefix /p → host bits = 32−p.
Addresses in block = 2^(32−p).
First address = network; last = broadcast (IPv4 subnet).
For fragmentation, fragment payload sizes must respect the outgoing MTU and, except the final fragment, be multiples of 8 bytes. MF=1 means more fragments follow.
```

METHOD

Routing: DV vs LS

```
Distance Vector → Bellman-Ford recurrence:
D_x(y) = min_v { c(x,v) + D_v(y) }
Link State → build a complete topology view, then run Dijkstra from the source.
RIP → distance-vector, hop count metric, max usable hop count 15.
OSPF → link-state, Dijkstra, cost metric.
BGP → inter-domain path-vector protocol.
```

METHOD

TCP sequence / ACK

```
TCP sequence number identifies the first byte in a segment's payload.
ACK number = next byte expected (cumulative ACK).
SYN and FIN each consume one sequence number even though they carry no application data.
If segment starts at seq S and carries L bytes, next expected byte is S+L.
```

METHOD

TCP timeout / RTO

```
Smoothed RTT: SRTT = (1−α)SRTT + α·SampleRTT.
RTT variation: RTTVAR = (1−β)RTTVAR + β|SRTT−SampleRTT|.
Typical RTO form: SRTT + 4·RTTVAR.
Do not use a retransmitted sample as an ordinary RTT sample (Karn's rule).
```

METHOD

DNS / HTTP timing

```
DNS maps names to records; ordinary DNS queries commonly use UDP 53, while TCP 53 is used when required (e.g. zone transfer).
HTTP is application-layer request/response.
For timing questions, count each required RTT separately and add transmission/processing time only when the question gives it.
Persistent connections avoid repeated TCP handshakes; pipelining can overlap requests.
```

METHOD

Delay / throughput

```
Transmission = packet bits / link rate.
Propagation = distance / propagation speed.
Store-and-forward across H links: H·(transmission + propagation) when all links have the same values and processing/queueing are ignored.
BDP = bandwidth × RTT.
```

1. Bit stuffing

METHOD NOTES

Suppose the payload contains **1111101**. What bit-oriented stuffing occurs?

After the first five consecutive 1s, the sender inserts a 0: **11111001**. The receiver removes that inserted 0 after detecting five consecutive 1s.

Trap: the inserted 0 is not application data.

2. CIDR block size

METHOD NOTES

How many IPv4 addresses are in **192.168.10.0/26**?

Host bits = 32−26 = 6. Therefore addresses = 2^6 = **64**. The block spans 192.168.10.0 through 192.168.10.63; in the usual subnet interpretation, .0 is the network address and .63 the broadcast address.

3. TCP sequence and ACK

METHOD NOTES

A TCP segment has sequence number 1000 and carries 500 bytes of data. What cumulative ACK should the receiver send if all bytes through the segment arrived correctly?

The bytes are 1000 through 1499, so the next expected byte is **1500**. ACK = **1500**.

If the segment instead had SYN set and carried no data, the SYN would still consume one sequence number.

4. RTT/RTO update

METHOD NOTES

Let previous SRTT = 100 ms, α = 0.125, and a new SampleRTT = 140 ms. Compute the new SRTT.

SRTT = 0.875×100 + 0.125×140 = 87.5 + 17.5 = **105 ms**. In a full RTO calculation, RTTVAR must also be maintained; do not replace RTO with SRTT alone.

5. HTTP persistent timing

METHOD NOTES

A page and 3 embedded objects are on the same server. RTT = R and transmission time is negligible. Under persistent HTTP with pipelining, after the TCP connection is established, requests can be sent together.

Using the standard simplified GATE model: TCP handshake = R, base-object request/response = R, pipelined embedded-object requests/responses = R. Total ≈ **3R**.

Trap: if the problem explicitly includes DNS lookup or TLS handshake, add those RTTs according to the stated model.

6. Distance Vector update

METHOD NOTES

Router X has neighbor A with link cost 2. A advertises distance 5 to destination Y. What candidate distance does X obtain through A?

Bellman-Ford candidate = c(X,A) + D_A(Y) = 2 + 5 = **7**. X compares this with all other neighbors and chooses the minimum candidate.

⚡ CN Last-Minute Recall

METHOD

- Bit stuffing: insert **0 after five consecutive 1s**.
- /p IPv4 block size = **2^(32−p)** addresses.
- Fragment offset is measured in **8-byte units**; MF=1 means more fragments follow.
- DV = Bellman-Ford; LS = Dijkstra. RIP = DV, OSPF = LS, BGP = path vector.
- TCP ACK = **next byte expected**; SYN/FIN each consume one sequence number.
- BDP = bandwidth × RTT.
- SMTP sends mail; POP3/IMAP retrieve it.
- DNS A/AAAA = address, CNAME = alias, MX = mail server, PTR = reverse lookup.

9 Last-minute one-liners

LAST-MINUTE · FORMULAS & RULES

- MSS = MTU − IP − TCP headers = 1460 B for a 1500-B MTU. TCP header 20–60 B; UDP header 8 B.
- RTO = SRTT + 4·RTTVAR; SRTT uses α = 1/8, RTTVAR uses β = 1/4. Karn: skip RTT samples from retransmissions, back off exponentially.
- Transmission delay = packet size / link rate; propagation delay depends on distance and propagation speed.
- Throughput is limited by the bottleneck link/rate along the path, not simply by the fastest link.
- **Subnet IPv4 address:** network prefix + host part. Increasing prefix length reduces hosts per subnet.
- TCP flow control is receiver-window based; congestion control is network-condition based. Do not conflate the two.
- **Sliding window efficiency (a = Tp/Tt):** stop-and-wait 1/(1+2a); Go-Back-N and Selective Repeat N/(1+2a), capped at 1. Window for 100% = 1+2a; GBN needs sequence space N+1, SR needs 2N.
- Shannon $C = B \cdot \log_2(1 + S/N)$ with $S/N = 10^{dB/10}$; Nyquist $C = 2B \cdot \log_2 L$. Use the smaller when both apply.
- Token bucket burst time $S = b/(M - r)$; a leaky bucket outputs at a constant rate and drops overflow.

LAST-MINUTE · MUST REMEMBER

- **ARP:** IP→MAC on the same LAN (broadcast request, unicast reply). Across routers: MAC changes every hop, IP stays the same.
- DHCP = DORA over UDP (server 67, client 68). ICMP = errors/diagnostics, IP protocol 1; traceroute uses TTL expiry.
- SYN and FIN each consume one sequence number; ACK number = next byte expected; 3 duplicate ACKs trigger fast retransmit.
- OSI is a reference model; TCP/IP is the practical protocol suite used on the Internet.
- CRC detects errors using polynomial division; the receiver checks whether the received polynomial is divisible by the generator.
- CSMA/CD belongs to traditional shared Ethernet; modern switched full-duplex Ethernet does not use collision detection.
- CIDR uses prefix notation and enables route aggregation; longest-prefix matching selects the most specific matching route.
- TCP is connection-oriented and reliable; UDP is connectionless and does not provide TCP-style reliability.
- DNS maps names to records such as A/AAAA, MX, NS, and CNAME; DNS commonly uses UDP, with TCP also used when required.
- HTTP is application-layer; HTTPS is HTTP carried over TLS.
- Router forwarding uses the longest matching prefix, not the first match. Usable hosts in a /n block = $2^{32-n} - 2$; VLSM allocates the biggest subnet first.

LAST-MINUTE · COMMON TRAPS

- **Bit stuffing:** insert 0 after FIVE consecutive 1s (flag 01111110). A switch has one collision domain per port but ONE broadcast domain.
- Fragmentation can occur in IPv4 when a packet exceeds an outgoing link MTU; fragmentation fields include Identification, Flags, and Fragment Offset.
- **Tahoe vs Reno:** three duplicate ACKs → Tahoe cwnd = 1, Reno cwnd = ssthresh + 3 (fast recovery). A timeout gives cwnd = 1 in both.
- **HTTP is stateless:** cookies add state. Conditional GET answers 304 Not Modified with no body. IPv4 header checksum covers the header only and changes at each hop because of TTL.

LAST-MINUTE · GATE FAVOURITES

- **HTTP RTT counting:** non-persistent serial = 2R per object; persistent = R + one R per object; persistent + pipelined = 3R.
- Know slow start, congestion avoidance, fast retransmit, and fast recovery at the level needed to trace cwnd evolution.